

CMSC 472 - Introduction to Deep Learning

Final Project Instructions & Grading Rubric

Deadline: 11:59 PM May 08, 2026.

Total: 80 points + up to 5 bonus points.

1 Overview

The final project asks you to carry out a complete deep learning research cycle: identify a problem, design and implement an approach, run rigorous experiments, and communicate your findings in a paper-style report. Projects are evaluated on both the *quality of ideas* and the *rigor of execution*.

2 What to submit?

- **Written Report** — NeurIPS format, 8 pages max (excluding references). Due 11:59 PM, May 08, 2026.
- **Code** — Upload it as a ZIP file to Gradescope alongwith a README. Due 11:59 PM, May 08, 2026.

Note: Both deliverables must be submitted together as a .zip file. A report without runnable code, or code without a report, will result in loss of points.

3 Written Report

Use the NeurIPS L^AT_EX template. The report should be self-contained and written for a reader who has general Deep learning knowledge but no prior familiarity with your specific project.

Required Sections

1. **Introduction** — State the problem, motivate why it matters, and summarize your contributions.
2. **Related Work** — Situate your work relative to at least 5 relevant papers. Explain what differentiates your approach.
3. **Method** — Describe your approach clearly enough to be reproduced. Include model diagrams or pseudocode where helpful.
4. **Experiments** — Describe datasets, baselines, metrics, and experimental setup. Present results with appropriate statistical context (std dev, confidence intervals, or multiple runs).

5. **Analysis** — At least one analysis experiment that provides insight into *why* your approach works (or doesn't).
6. **Discussion** — Reflect on limitations, failure modes, and future directions.
7. **Conclusion** — Summarize findings concisely.

Important Report Tips

- Every claim should be supported by a citation or an experiment.
- Tables and figures must have captions that are self-explanatory.
- Do not pad with unnecessary background. 8 pages should be earned, not filled.

4 Code

Your submission must include everything needed to reproduce the key results in your paper.

Required Directory Structure

```
project/
|-- README.md          # Setup, dependencies, and run instructions
|-- requirements.txt    # or environment.yml
|-- data/              # Download scripts or instructions (not raw data if large)
|-- src/               # Source code (.py), well-organized
|-- experiments/        # Scripts to reproduce each key table/figure
|-- results/           # Saved outputs, logs, or model checkpoints
|-- notebooks/         # optional
```

README Must Include

- One-line project description
- Environment setup instructions (`pip install -r requirements.txt` or equivalent)
- Commands to reproduce each main result in the paper
- Expected runtime and hardware (e.g., “~2 hrs on a single A100”)

Code Quality Expectations

- Modular, readable code with documentation
- No hardcoded paths except in the current directory; use config files or argument parsers
- Reproducible random seeds set and documented

5 Grading Rubric (80 Points)

Part 1 — Written Report (50 points)

1A. Problem Formulation & Motivation (10 pts)

Score	Rubric
9–10	Problem is precisely defined, strongly motivated with empirical evidence or clear real-world impact. Scope is well-calibrated for the project timeline.
7–8	Problem is clearly stated and motivation is reasonable, though some aspects could be sharper.
4–6	Problem is vague or motivation is superficial (“it would be interesting to...”). Scope may be too narrow or too broad.
0–3	Problem is ill-defined or barely motivated.

1B. Technical Approach & Novelty (16 pts)

Score	Rubric
14–16	Approach is technically sound and demonstrates a meaningful novel contribution — a new method, an adaptation to a new domain, an insightful combination of existing techniques, or a new dataset/benchmark. Clearly differentiated from prior work.
10–13	Approach is solid and includes some novel element, but the contribution is incremental or differentiation from existing work is partly unclear.
5–9	Approach is a straightforward application of known methods with minimal novelty. Related work comparison is weak.
0–4	Approach is a near-copy of a baseline with no discernible contribution, or technically flawed.

Note: Negative results are acceptable if the paper provides a thorough analysis of *why* an approach failed.

1C. Experiments & Evaluation (16 pts)

Score	Rubric
14–16	Experiments directly support all claims. Baselines are appropriate. Includes at least one diagnostic experiment.
10–13	Most claims are backed by experiments. Baselines are reasonable. Minor gaps (e.g., only one run, missing diagnostics).
5–9	Experiments are incomplete or weak — e.g., no baselines, or results cherry-picked.
0–4	Experiments are absent or fundamentally flawed (e.g., data leakage, test set used for model selection).

1D. Writing Quality & Clarity (8 pts)

Score	Rubric
7–8	Clear, precise, and well-organized. Figures and tables are clear. A reader can follow the paper without referring to the code.
5–6	Generally readable with minor clarity or organization issues.
3–4	Significant clarity issues — key details missing, poor figure quality, or confusing structure.
0–2	Hard to follow; major sections are missing or incoherent.

Part 2 — Code & Reproducibility (30 points)

2A. Reproducibility (15 pts)

Score	Rubric
13–15	Main results reproduce within reasonable variance with provided scripts. Random seeds fixed. Hardware/runtime documented.
9–12	Most results reproduce with minor manual effort. Small discrepancies from the paper.
5–8	Results partially reproduce; some scripts are broken or missing.
0–4	Code does not run or results are far from reported numbers with no explanation.

2B. Code Quality & Documentation (10 pts)

Score	Rubric
9–10	Well-organized, readable code. Functions documented. No dead code. README is thorough.
6–8	Mostly clean code with a functional README. Minor issues (some hardcoded paths other than current directory, sparse comments).
3–5	Code works but is difficult to follow. README is minimal or missing key steps.
0–2	Code is disorganized, poorly commented, or the README is absent.

2C. Alignment Between Code and Paper (5 pts)

Score	Rubric
5	Every architecture/hyperparameter/dataset detail in the paper matches the code exactly, or discrepancies are explicitly noted.
3–4	Minor inconsistencies between paper and code.
0–2	Significant discrepancies that call reported results into question.

Part 3 — Bonus: Moon-Shot Attempt (up to +5 pts)

Up to **5 bonus points** are awarded if the team attempted an ambitious stretch goal (as described in the project proposal) and provides a clear write-up of what was tried, what happened, and what was learned — regardless of whether it succeeded. These points are not guaranteed by merely attempting something hard; the write-up must demonstrate thoughtful execution and genuine insight.

6 Point Summary

Component	Points
1A. Problem Formulation & Motivation	10
1B. Technical Approach & Novelty	16
1C. Experiments & Evaluation	16
1D. Writing Quality & Clarity	8
2A. Reproducibility	15
2B. Code Quality & Documentation	10
2C. Code–Paper Alignment	5
Total	80
Bonus: Moon-Shot Attempt	+5
Maximum	85

7 Academic Integrity

- All code must be written by your team. Clearly cite any code adapted from public repositories.
- Use of LLM tools (e.g., GitHub Copilot, ChatGPT) is permitted for coding assistance but must be disclosed in the report. You are responsible for the correctness of all code.
- All text in the report must be your own. LLM-generated prose is not permitted without disclosure.

Questions?

Post on Piazza or visit office hours before the deadline. Last-minute questions about submission format may not receive a response.